# Effect of Pixel Shuffling on Fully Connected Neural Network Performance

Priansh Yajnik (B24318)

07.02.2026

## 1 Introduction

This report investigates how pixel shuffling affects the performance of Fully Connected Neural Networks (FCNNs) on the MNIST dataset. The central question addressed is: While FCNNs are known to be independent of input ordering and lack spatial awareness, how does destroying pixel correlation through shuffling affect their learning performance?

## 2 Experimental Setup

Two experiments were conducted using PyTorch to implement a Fully Connected Neural Network with the following architecture:

- Input layer: 784 neurons (28×28 pixels)
- Hidden layer 1: 128 neurons with ReLU activation
- Hidden layer 2: 128 neurons with ReLU activation
- Hidden layer 3: 128 neurons with ReLU activation
- Output layer: 10 neurons (one for each digit)

The optimizer used was Adam with a learning rate of 0.001, and Cross-Entropy Loss was used as the loss function.

### 2.1 Experiment 1: Normal MNIST with Min-Max Scaling

In the first experiment:

- The MNIST dataset was used without any pixel shuffling
- Min-Max scaling was applied to normalize pixel values to the range [-1, 1]
- The model was trained for 10 epochs

**Results:**

- Training accuracy: 98%
- Testing accuracy: 97%

### 2.2 Experiment 2: Shuffled MNIST without Min-Max Scaling

In the second experiment:

- The pixel positions in the MNIST dataset were randomly shuffled
- The same shuffling pattern was applied to both training and testing data
- Pixel values were scaled by a factor of 10
- No min-max normalization was applied
- The model was trained for 100 epochs

**Results:**

- Training accuracy: 97.33%
- Testing accuracy: 96.12%

## 3 Key Observations

### 3.1 Effect of Training Duration

The most striking observation is the difference in convergence speed:

- **Normal MNIST:** Achieved 98% training accuracy in 10 epochs
- **Shuffled MNIST:** Required 100 epochs to achieve 97.33% training accuracy

Despite achieving similar final accuracies, the shuffled dataset required 10 times more epochs to converge. This dramatic difference reveals something important about how FCNNs learn.

### 3.2 Effect of Scaling

Two important observations regarding scaling were made:

1. **Min-Max scaling on shuffled data:** When min-max scaling (to the range [-1, 1]) was applied to the shuffled dataset, it destroyed the signal carried by the pixels, leading to significantly lower accuracy.
2. **Large-scale multiplication:** When pixels were scaled by large factors (100 or 1000), the accuracy deteriorated. This occurred because the weights had to decrease significantly to compensate for the large input values, which slowed down the loss reduction per epoch despite using the powerful Adam optimizer.

## 4 Understanding the Phenomenon: Spatial Arrangement vs. Weight Updates

While it may seem contradictory that an FCNN, which treats all inputs equally and lacks spatial awareness, performs differently on shuffled data, the explanation lies in understanding the difference between *spatial arrangement* and *weight update dynamics*.

## 4.1 What FCNNs Are Independent Of

FCNNs are independent of **spatial arrangement** in the sense that:

- There is no architectural constraint that enforces neighboring pixels to be processed together
- The model can theoretically learn any mapping from input to output, regardless of how pixels are arranged
- Given enough training time, an FCNN can learn to classify shuffled images just as well as normal images

Mathematically, if we have an input vector $\mathbf{x} = [x_1, x_2, \ldots, x_{784}]$ and a permuted version $\mathbf{x}' = [x_{\pi(1)}, x_{\pi(2)}, \ldots, x_{\pi(784)}]$ where $\pi$ is some permutation, the FCNN can learn different weight matrices $\mathbf{W}$ and $\mathbf{W}'$ such that:

$$f(\mathbf{x}; \mathbf{W}) = f(\mathbf{x}'; \mathbf{W}')$$

where $f$ represents the network's function.

## 4.2 What FCNNs Are NOT Independent Of

However, FCNNs are **not** independent of the *correlation structure* during the **weight update process**. This is the key distinction.

### 4.2.1 Role of Pixel Correlation in Weight Updates

In the normal MNIST dataset, neighboring pixels are highly correlated. For example, if pixel $(i, j)$ is part of a vertical stroke, then pixels $(i-1, j)$, $(i+1, j)$ are likely to have similar values. This correlation structure means that when the network updates weights, it can exploit these statistical regularities.

Consider the weight update rule in gradient descent:

$$w_{ij}^{(t+1)} = w_{ij}^{(t)} - \eta \frac{\partial L}{\partial w_{ij}}$$

where:

- $w_{ij}$ is the weight connecting input $j$ to neuron $i$
- $\eta$ is the learning rate
- $L$ is the loss function

The gradient can be expanded using the chain rule:

$$\frac{\partial L}{\partial w_{ij}} = \frac{\partial L}{\partial a_i} \cdot \frac{\partial a_i}{\partial w_{ij}} = \frac{\partial L}{\partial a_i} \cdot x_j$$

where $a_i = \sum_j w_{ij} x_j$ is the activation before the nonlinearity.

### 4.2.2 Why Correlation Matters

When pixels are correlated (as in normal MNIST), the gradient updates for different weights exhibit certain patterns:

1. **Consistent gradient directions:** If pixels $x_j$ and $x_k$ are highly correlated (often both large or both small together), their corresponding weight updates tend to move in consistent directions across different training samples.

2. **Reinforcement of patterns:** When the network sees multiple examples where correlated pixels co-occur (e.g., vertical strokes in digit "1"), the weights connecting to these pixels receive reinforcing gradient signals, leading to faster convergence.

3. **Efficient feature learning:** The network can learn meaningful features more quickly because the statistical structure in the data provides a "guide" for which weights should be updated together.

Mathematically, the expected gradient over the data distribution $\mathbb{E}[\frac{\partial L}{\partial w_{ij}}]$ benefits from the covariance structure of the inputs:

$$\text{Cov}(x_j, x_k) = \mathbb{E}[(x_j - \mu_j)(x_k - \mu_k)]$$

When $\text{Cov}(x_j, x_k)$ is high (as with neighboring pixels), the weight updates for $w_{ij}$ and $w_{ik}$ contain complementary information that helps the network learn faster.

### 4.2.3 What Happens with Shuffled Pixels

When pixels are shuffled:

1. **Destroyed correlation structure:** Pixels that were once neighbors are now scattered randomly across the input vector. The correlation between adjacent positions in the input vector is now essentially zero or random.

2. **Noisier gradient signals:** The weight updates no longer benefit from the natural correlation structure. Each weight must be learned more independently, without the "help" of correlated gradient signals from nearby pixels.

3. **Slower convergence:** The network can still learn (given enough time), but the learning process is less efficient. The optimizer must work harder to find the right combination of weights because the statistical structure that would normally guide learning has been removed.

## 4.3 A Simple Mathematical Example

Consider a simplified case with just three pixels: $x_1, x_2, x_3$ and weights $w_1, w_2, w_3$ connecting to a single neuron.

**Case 1: Correlated pixels (normal MNIST)**

Suppose $x_1$ and $x_2$ are part of the same stroke and are highly correlated:

- When $x_1 = 200$, typically $x_2 \approx 180$
- When $x_1 = 50$, typically $x_2 \approx 45$

The gradients $\frac{\partial L}{\partial w_1} \propto x_1$ and $\frac{\partial L}{\partial w_2} \propto x_2$ will vary together across different samples. This means:

- If the network needs to increase $w_1$ to reduce loss, it likely also needs to increase $w_2$
- The optimizer receives consistent signals across batches
- Convergence is faster

**Case 2: Shuffled pixels**
After shuffling, $x_1$ and $x_2$ might come from completely different parts of the original image:

- $x_1$ might be from the top-left corner
- $x_2$ might be from the bottom-right corner
- Their values are essentially uncorrelated

Now the gradients $\frac{\partial L}{\partial w_1}$ and $\frac{\partial L}{\partial w_2}$ vary independently:

- The optimizer cannot use the correlation to guide weight updates
- Each weight must be learned more independently
- More iterations are needed to find the optimal weight configuration

# 5 Why the Scaling Factor Matters

## 5.1 Min-Max Scaling on Shuffled Data

When min-max scaling was applied to the shuffled data, it destroyed the signals from the pixels. This is because:

- The original pixel values in MNIST range from 0 to 255
- Scaling to [-1, 1] compresses this range dramatically
- For shuffled pixels, which already lack spatial correlation, this compression removes important magnitude information
- The network struggles to distinguish between meaningful signal and noise

## 5.2 Large-Scale Multiplication

When pixels were multiplied by large factors (100 or 1000):

$$\mathbf{x}_{\text{scaled}} = k \cdot \mathbf{x}, \quad k \in \{100, 1000\}$$

The network faced the following issue:

1. The weighted sum becomes very large: $a_i = \sum_j w_{ij}(k \cdot x_j) = k \sum_j w_{ij} x_j$

2. To keep the activations in a reasonable range, the weights must become very small: $w_{ij} \propto \frac{1}{k}$
3. The gradient with respect to weights becomes: $\frac{\partial L}{\partial w_{ij}} = \frac{\partial L}{\partial a_i} \cdot (k \cdot x_j)$
4. While the gradient magnitude increases, the weight updates must be very small to avoid instability
5. Even with adaptive optimizers like Adam, which adjust learning rates, the overall convergence is slower because the weight space the optimizer must explore has a very different scale

This demonstrates that while FCNNs are theoretically capable of learning with any input scaling, the practical convergence speed is heavily affected by the scale of the inputs and how well that scale matches the initialization and optimization dynamics.

# 6 Conclusion

This experiment demonstrates a subtle but important distinction:

- **Spatial arrangement independence:** FCNNs do not inherently require inputs to be spatially organized. They can learn to classify shuffled images given sufficient training time.
- **Correlation dependence during learning:** However, the *speed* of learning is highly dependent on the correlation structure between input features. When pixels that naturally co-occur (are part of the same visual patterns) are presented together, the weight update process is more efficient.
- **Weight updates exploit correlation:** The weight update mechanism in gradient descent benefits from correlated inputs because the gradients provide consistent, reinforcing signals. Destroying this correlation by shuffling pixels makes learning slower, even though the final accuracy can be similar.

In summary, while FCNNs treat all input positions equally from an architectural perspective, the learning dynamics—specifically how weights are updated during backpropagation—are significantly affected by the statistical correlation structure of the inputs. This explains why the same FCNN architecture requires 10 times more epochs to achieve similar performance on shuffled versus normal MNIST data.